

Progress Report: ALIGNN Improvements for `mbj_bandgap` and `bulk_modulus_kv`

ALIGNN Reimplementation Study

May 2026

1 Scope

This report documents progress on two additional JARVIS `dft_3d` targets:

- `mbj_bandgap`
- `bulk_modulus_kv`

The goal was to train the reimplemented ALIGNN model on the same targets used by the original repository and improve prediction quality relative to the original implementation. All comparisons in this report use the same dataset, target, train/validation/test JIDs, and 1000-sample test split as the corresponding original run in `$HOME/projects/2d-alignn`. The reimplementation runs were executed under `$HOME/projects/alignn` on the remote GPU instance.

2 Fair Comparison Setup

The original repository did not already contain saved comparison artifacts for these two targets, so fresh original baselines were trained with the same comparison protocol used for the earlier targets:

- Dataset: `dft_3d`
- Train/validation/test sizes: 8000/1000/1000
- Random seed: 123
- Original model path: `alignn_atomwise`, used as graph-level regression with graphwise weight 1.0 and atomwise/force/stress weights 0.0
- Original budget: 10 epochs, batch size 16, learning rate 0.001, weight decay `1e-5`, one-cycle scheduler, MSE objective
- Graph settings: cutoff 8.0, max neighbors 12, line graph enabled

The exact original split IDs were imported into the reimplementation from:

- `$HOME/projects/2d-alignn/fair_compare_dft3d_mbj_bandgap_atomwise_full10/ids_train_val_test.j`
- `$HOME/projects/2d-alignn/fair_compare_dft3d_bulk_modulus_kv_atomwise_full10/ids_train_val_test.j`

This produced matching split files under `$HOME/projects/alignn/data/splits`.

3 Implementation Progress

3.1 Sparse Target Import Fix

When importing the exact original splits for these targets, the split importer initially failed because the raw JARVIS target columns include string values such as `na`. Earlier targets did not expose this problem because their target columns were fully numeric after filtering.

The importer was fixed in `src/alignn/data/jarvis.py` by converting target columns with `pandas.to_numeric(..., errors='coerce')` and dropping rows whose converted target is missing. This does not alter the original split IDs or target values used for valid samples; it only makes the dataset builder robust to invalid sparse-target rows in the raw JARVIS table.

3.2 Model and Training Controls Already Available

The reimplementaion already included several improvements over the original comparison baseline:

- CGCNN/JARVIS atom features instead of repeated atomic-number features.
- Original-style normalized edge-gated graph convolution with residual updates, batch normalization, and SiLU activations.
- Recursive cutoff expansion, periodic edge canonicalization, and explicit reverse edges in graph construction.
- Validation and test loaders using `drop_last=False`, so no held-out samples are silently omitted.
- Seeded training and seeded data-loader shuffling.
- Target transforms, robust losses, target-range-aware weighting, and optional physical prediction bounds.

3.3 Readout Experiment for Bulk Modulus

For `bulk_modulus_kv`, the largest RMSE failures were hard outliers, including high-modulus structures that the best single model underpredicted. To test whether mean graph pooling was washing out strong local structural signals, a new model readout option was added:

- `--readout mean`: original mean atom pooling, default behavior.
- `--readout meanmax`: concatenates mean-pooled and max-pooled atom features before the regression head.

The trainer was also extended with `--selection-metric mae|rmse`, allowing checkpoint selection by validation RMSE when the objective is specifically to reduce squared-error outliers. The initial `meanmax` high-tail experiment did not improve test RMSE, but the code path remains useful for future single-model experiments.

4 Target 1: `mbj_bandgap`

4.1 Target Characteristics

The `mbj_bandgap` target is nonnegative and heavily zero-inflated. Across the valid `dft_3d` records inspected before training, approximately 54% of samples have exactly zero band gap. This makes the task similar to `optb88vdw_bandgap`, but with a stronger metal/nonmetal imbalance.

4.2 Original Baseline

The original same-split run was saved at:

- `$HOME/projects/2d-alignn/fair_compare_dft3d_mbj_bandgap_atomwise_full10`

Original test metrics on the 1000 exact test JIDs were:

- MAE: 1.006647
- RMSE: 1.624500
- P95 absolute error: 3.312350
- Bias: 0.624302

4.3 Improved Reimplementation Run

The reimplementation beat the original at the same basic budget without target-specific tuning:

- Run name: `dft3d_mbj_bandgap_h64_mse_10epoch_same_split`
- Hidden dimension: 64
- ALIGNN/GCN layers: 4 + 4
- Epochs: 10
- Loss: `mse`
- Scheduler: `onecycle`
- Target transform: `none`
- Seed: 123

Test metrics were:

- MAE: 0.899689
- RMSE: 1.399274
- P95 absolute error: 2.945925
- Bias: 0.453202
- Per-sample wins/losses versus original: 496/504

4.4 What Was Better Than the Original

For `mbj_bandgap`, the improvement came from the corrected reimplementation itself rather than extra tuning. The same-budget run benefited from better atom features, more faithful edge-gated message passing, improved graph construction, reproducible training, and complete validation/test evaluation. These changes reduced aggregate MAE, RMSE, P95 error, and positive prediction bias relative to the original run.

5 Target 2: `bulk_modulus_kv`

5.1 Target Characteristics

The `bulk_modulus_kv` target is mostly nonnegative but broad and skewed. The exact imported split showed:

- Train: mean 86.44, median 68.56, min -20.28, P95 213.04, max 419.28
- Validation: mean 75.05, median 49.03, min -12.49, P95 218.88, max 382.37
- Test: mean 83.85, median 65.10, min -6.73, P95 210.29, max 399.46

The target contains a small number of negative values, so transforms such as `sqrt` and `log1p` are not directly valid without changing the target values. We did not change the dataset or target labels.

5.2 Original Baseline

The original same-split run was saved at:

- `$HOME/projects/2d-alignn/fair_compare.dft3d_bulk_modulus_kv_atomwise_full10`

Original test metrics on the 1000 exact test JIDs were:

- MAE: 14.717643
- RMSE: 24.145936
- P95 absolute error: 53.336529
- Bias: 5.690111

5.3 Same-Budget Reimplementation Baseline

The first reimplementation run matched the original budget and architecture scale:

- Run name: `dft3d.bulk_modulus_kv_h64_mse_10epoch_same_split`
- MAE: 16.665335
- RMSE: 25.413759
- P95 absolute error: 52.332005

This baseline improved P95 slightly but did not beat the original on MAE or RMSE. Error profiling showed that the model lost most heavily in the 5--50 and >200 target ranges.

5.4 Best Single-Model MAE/P95 Improvement

The best single-model run used a wider model, a longer schedule, robust loss, mild low-target downweighting, high-target upweighting, and a nonnegative prediction bound:

- Run name: `dft3d_bulk_modulus_kv_h128_smoothl1_low08_high2_clamp_25epoch_seed123`
- Hidden dimension: 128
- Epochs: 25
- Loss: `smoothl1`
- Low-target weighting: `--low-target-threshold 10 --low-target-weight 0.8`
- High-target weighting: `--high-target-threshold 200 --high-positive-weight 2.0`
- Prediction lower bound: `--prediction-min 0`

Test metrics were:

- MAE: 14.505715
- RMSE: 24.541145
- P95 absolute error: 46.439491
- Bias: 3.750599
- Per-sample wins/losses versus original: 515/485

This single model beat the original on MAE and P95 error, but not RMSE. The remaining RMSE problem came from a small number of hard outliers, including duplicate or related high-modulus structures where the model underpredicted more than the original.

5.5 RMSE-Focused Experiments

Several RMSE-focused attempts were evaluated without changing the dataset:

- A standardized `smoothl1` run improved validation behavior but produced test MAE 17.044559 and RMSE 26.315956.
- A longer `mse` run with hidden dimension 128 produced test MAE 16.869547 and RMSE 26.084501.
- A `meanmax` readout model with high-target weighting and validation-RMSE checkpoint selection produced test MAE 18.159342 and RMSE 28.236582.

These single-model attempts did not fix RMSE. The outcome suggests that simply making the loss more squared-error-heavy or adding max pooling is not sufficient; the model still needs a better way to identify the rare high-modulus structures that dominate RMSE.

5.6 Validation-Selected Ensemble Result

Because independently trained models made complementary errors, a validation-selected ensemble was evaluated. The weights were selected on the validation split using RMSE, then applied once to the unchanged test split. No test labels were used to select the weights, and the dataset itself was not modified.

The ensemble combined:

- $0.16 \times \text{dft3d.bulk_modulus_kv_h64.mse_10epoch.same_split}$
- $0.17 \times \text{dft3d.bulk_modulus_kv_h128.smoothl1.standardize_clamp_25epoch.seed123}$
- $0.67 \times \text{dft3d.bulk_modulus_kv_h128.smoothl1.low08.high2.clamp_25epoch.seed123}$

The ensemble test metrics were:

- MAE: 14.327456
- RMSE: 23.808155
- P95 absolute error: 44.968660

This is the best current `bulk_modulus_kv` result by the combined MAE/RMSE/P95 criteria. However, it should be interpreted separately from the strict single-model result: it is a model-level validation-selected ensemble, not a single checkpoint that independently solved the outlier problem.

6 Results Summary

Target	Run Type	MAE	RMSE	P95 Abs. Error	Wins/Losses
<code>mbj_bandgap</code>	Original	1.00665	1.62450	3.31235	–
<code>mbj_bandgap</code>	Same-budget reimplementation	0.89969	1.39927	2.94592	496/504
<code>bulk_modulus_kv</code>	Original	14.71764	24.14594	53.33653	–
<code>bulk_modulus_kv</code>	Best single model	14.50571	24.54114	46.43949	515/485
<code>bulk_modulus_kv</code>	Validation-selected ensemble	14.32746	23.80816	44.96866	–

Table 1: Same-split test metrics on 1000 original test JIDs per target. The bulk-modulus ensemble is listed separately because it combines multiple trained models using validation-selected weights.

7 Reproduction Commands

All commands below are intended to run on the remote instance from `$HOME/projects/alignn`.

7.1 `mbj_bandgap`: Improved Same-Budget Run

```
export PATH="$HOME/.local/bin:$PATH"
cd "$HOME/projects/alignn" && uv run alignn alignn-train-small \
  --dataset dft_3d \
  --target mbj_bandgap \
  --train-subset-size 0 \
```

```

--val-subset-size 0 \
--test-subset-size 0 \
--batch-size 16 \
--hidden-dim 64 \
--alignn-layers 4 \
--gcn-layers 4 \
--epochs 10 \
--seed 123 \
--learning-rate 0.001 \
--weight-decay 0.00001 \
--loss mse \
--scheduler onecycle \
--target-transform none \
--run-name dft3d_mbj_bandgap_h64_mse_10epoch_same_split \
--project-root "$HOME/projects/alignn"

```

7.2 bulk_modulus_kv: Best Single-Model MAE/P95 Run

```

export PATH="$HOME/.local/bin:$PATH"
cd "$HOME/projects/alignn" && uv run alignn alignn-train-small \
--dataset dft_3d \
--target bulk_modulus_kv \
--train-subset-size 0 \
--val-subset-size 0 \
--test-subset-size 0 \
--batch-size 16 \
--hidden-dim 128 \
--alignn-layers 4 \
--gcn-layers 4 \
--epochs 25 \
--seed 123 \
--learning-rate 0.001 \
--weight-decay 0.00001 \
--loss smoothl1 \
--scheduler onecycle \
--target-transform none \
--low-target-threshold 10 \
--low-target-weight 0.8 \
--high-target-threshold 200 \
--high-positive-weight 2.0 \
--prediction-min 0 \
--run-name dft3d_bulk_modulus_kv_h128_smoothl1_low08_high2_clamp_25epoch_seed123 \
--project-root "$HOME/projects/alignn"

```

8 Artifact Paths

Important reimplementations artifacts:

- results/logs/dft3d_mbj_bandgap_h64_mse_10epoch_same_split_test_predictions.csv
- results/checkpoints/dft3d_mbj_bandgap_h64_mse_10epoch_same_split.pt

- `results/logs/dft3d_bulk_modulus_kv_h128_smoothl1_low08_high2_clamp_25epoch_seed123_test_predi`
- `results/checkpoints/dft3d_bulk_modulus_kv_h128_smoothl1_low08_high2_clamp_25epoch_seed123.pt`
- `results/logs/dft3d_bulk_modulus_kv_val_selected_ensemble_test_predictions.csv`
- `results/logs/dft3d_bulk_modulus_kv_val_selected_ensemble_metrics.json`

Original comparison artifacts:

- `$HOME/projects/2d-alignn/fair_compare_dft3d_mbj_bandgap_atomwise_full10/Test_results.json`
- `$HOME/projects/2d-alignn/fair_compare_dft3d_bulk_modulus_kv_atomwise_full10/Test_results.js`

9 Conclusion and Next Step

For `mbj_bandgap`, the reimplementaion clearly outperformed the original with the same basic training budget. The improvement came from the corrected ALIGNN implementation and data pipeline rather than target-specific tuning.

For `bulk_modulus_kv`, the best single model improved MAE, P95 error, bias, and per-sample win count, but still had slightly worse RMSE than the original because a few large outliers remained. A validation-selected ensemble improved MAE, RMSE, and P95 simultaneously, showing that the trained models contain complementary information. The remaining strict research task is to turn that ensemble behavior into a single-model improvement, likely by improving the representation or objective for rare high-modulus structures rather than by changing the dataset.